

extracts reusable signals and trains PCU-Select once. $C_{\text{PCU}}^{\text{pair}}$ scores and selects data for one target pair. The first expression repeats datastore construction P times, whereas the second pays the offline cost once. Section 5 measures each cost term.

Lower cost matters only if the selected subset preserves target-specific quality. We express the quality difference from per-PEFT LESS as

$$\Delta_{\text{quality}} = \text{Perf}_{\text{PCU}} - \text{Perf}_{\text{LESS}}, \quad |\Delta_{\text{quality}}| \leq \epsilon, \quad (4)$$

where Perf_{PCU} and $\text{Perf}_{\text{LESS}}$ denote their downstream scores and ϵ is the acceptable difference. The cost equations also identify when the shared offline investment pays off. For a fixed Q , PCU-Select costs less once P exceeds $P^* = C_{\text{offline}} / [C_{\text{LESS}}^{\text{data}} + Q(C_{\text{LESS}}^{\text{rank}} - C_{\text{PCU}}^{\text{pair}})]$, assuming the denominator is positive. The objective is therefore to retain per-PEFT LESS quality while replacing repeated datastore construction with shared offline work.

Design requirements. The formulation yields four design requirements. **PEFT conditioning** includes p so the example ranking can change with the target configuration. **Task conditioning** uses V_t so the model learns task-specific value instead of generic example quality. **Structured PEFT code** records where and how each PEFT updates the model in a form shared across families. **Reusable execution** moves gradient extraction, label construction, and scorer training offline, leaving cached-feature scoring for each new (p, t) pair. The next section shows how PCU-Select meets these requirements.

4 Method: PCU-Select

PCU-Select learns the conditional utility surrogate in Eq. (2) and selects a budgeted subset for a target PEFT-task pair. Figure 2 summarizes two stages. Offline, it extracts reusable sample and task signals, constructs two types of utility label, and trains one scorer. Online, it combines cached sample features with a target (p^*, t^*) , scores the pool, and distributes budget across semantic clusters. Algorithms 1 and 2 specify both stages.

Shared Intervention Sites Enable Cross-PEFT Reuse

PEFT families expose parameter tensors with different shapes and roles, so their raw gradients cannot share one cache (Hu et al. 2022; Houlsby et al. 2019; Liu et al. 2022). PCU-Select instead uses a fixed set Ω of *intervention sites*. Each site is a selected module output through which a PEFT update affects the backbone. The set Ω is a shared coordinate system because sample gradients, task gradients, and each PEFT’s map of affected sites all use the same index ω .

PCU-Select hooks attention, MLP, and post-residual outputs at eight evenly spaced Transformer layers, giving $|\Omega| = 24$. A site’s response-loss gradient describes how a sample or task pushes that activation. By the chain rule, PEFT-parameter gradients factor through gradients at the module outputs they modify. Site gradients therefore avoid both full PEFT-parameter reconstruction and shape mismatches between fused and split projections. Post-residual sites place adapters in the same index space as LoRA and (IA)³.

A site index records only *where* a configuration acts. PCU-Select separately encodes operator and capacity features to record *how* it acts and how much it can update. This factorization reuses one sample signature without treating additive, multiplicative, and bottleneck updates as equivalent.

Each configuration maps to a site mask and operator type. Attention LoRA and (IA)³-attention map to attention-output sites. MLP LoRA and (IA)³-FFN map to MLP-output sites, while adapters map to block-residual sites. Prefix tuning maps to attention-output sites; BitFit maps to a bias shift. These mappings define each normalized site weight:

$$\begin{aligned} w_p^\omega &= \text{mask}(p, \omega) \kappa(p, \omega), \\ \kappa(p, \omega) &= \tanh(\eta \log(1 + \text{cap}(p, \omega)/d_{\text{model}})), \\ \tilde{w}_p^\omega &= w_p^\omega / \max(\varepsilon_w, \sum_{\omega'} w_p^{\omega'}). \end{aligned} \quad (5)$$

Here $\text{mask}(p, \omega)$ is zero at untouched sites, and $\text{cap}(p, \omega)$ measures trainable capacity. The width d_{model} normalizes capacity, η controls saturation through $\tanh$, and ε_w keeps the denominator nonzero. Thus, $\tilde{w}_p^\omega$ distributes unit weight over the sites used by p . Appendix Table 10 maps LoRA rank, adapter width, (IA)³ vector count, prefix length, and BitFit bias count to this scale. Operator type remains separate in z_p because a family-wide scalar would cancel under Eq. (5)’s normalization.

Shared Representations Preserve Target Specificity

Shared site indices make signals comparable, but the scorer must still distinguish each sample, task, and PEFT. PCU-Select uses three fixed-length vectors: the sample representation describes the example, the task representation describes the target need, and the PEFT representation describes where and how training acts. Site gradients supply separate label-construction signals.

The sample representation $z_x = [e_x; d_x; a_x] \in \mathbb{R}^{848}$ combines semantic, descriptive, and activation features. The 768-dimensional e_x is the frozen sentence-transformers/all-mpnet-base-v2 embedding of the instruction and response (Song et al. 2020; Reimers and Gurevych 2019). This encoder remains frozen during pool and target-benchmark processing. The 16-dimensional d_x records length, response-LM loss, entropy, and format. The 64-dimensional a_x summarizes response-token hidden states at the eight sampled layers.

Utility is task-relative, so PCU-Select forms $z_t \in \mathbb{R}^{848}$ by mean-pooling z_x over the task sketch’s 32 examples. For proxy labels, it separately averages their site gradients into g_t^ω . These 24×256 values remain outside z_t and serve only label construction.

The PEFT representation records the target trainable subspace. In $z_p = [m_p; c_p; r_p] \in \mathbb{R}^{128}$, the 24×4 mask m_p contributes 96 dimensions. Its channels mark active sites, additive updates, multiplicative scaling, and prefix-style control. Two 16-dimensional vectors encode capacity c_p and training recipe r_p . At this site resolution, q/v and q/k/v/o LoRA share attention-output sites, while capacity and recipe preserve their projection-level distinction.